CHAPTER 1: INTRODUCTION

DukaanSetu is a comprehensive multi-vendor e-commerce application developed using the **MERN stack** (MongoDB, Express.js, React.js, and Node.js). The platform's name, meaning "Shop Bridge," reflects its primary mission: connecting traditional offline shopkeepers with the modern, digital-first consumer base. In today's competitive retail landscape, small-scale local vendors often lack the technical resources to compete with global e-commerce giants. DukaanSetu levels the playing field by providing a robust digital infrastructure where shopkeepers can effortlessly digitize their inventory, manage orders, and monitor business growth through an intuitive dashboard.

For customers, the platform offers a streamlined shopping experience, allowing them to discover and purchase from neighborhood stores through advanced search and multi-criteria filtering. Key features include a persistent shopping cart, secure role-based authentication, and a unified order-level review system that fosters trust and transparency. Additionally, a centralized Admin panel ensures platform governance by standardizing product categories and providing real-time sales analytics. Ultimately, DukaanSetu is a community-centric ecosystem designed to revitalize local economies, empower small entrepreneurs, and provide modern convenience to neighborhood shoppers, effectively bridging the gap between physical storefronts and the digital marketplace.

CHAPTER 2: PROPOSED SYSTEM

1.Objective

- To provide a simple and robust digital infrastructure for traditional offline shopkeepers, allowing them to create an online presence, list their products, and reach a wider customer base without requiring advanced technical knowledge.
- To enable customers to easily find products from various local shops through advanced search and multi-criteria filtering (such as price, category, and shop location), making the neighborhood marketplace more accessible.
- To implement a unified shopping experience featuring a multi-vendor cart system, allowing customers to purchase items from different sellers in a single transaction while maintaining data integrity across the platform.
- To establish a clear and transparent order fulfillment process using real-time status updates (Pending vs. Delivered) and an integrated review system that allows customers to provide feedback on their overall order experience.
- To ensure a secure environment through robust JWT-based authentication and Role-Based Access Control (RBAC), ensuring that Admins, Shopkeepers, and Customers only access the features and data relevant to their specific needs.

2. Scope of the system

- Implementation of three distinct interfaces (Admin, Shopkeeper, Customer) with secure JWT-based authentication and specialized permissions for each role.
- A complete CRUD (Create, Read, Update, Delete) system for shopkeepers to list and manage their products, including descriptions, images, and pricing.
- Advanced searching and filtering capabilities allowing customers to find products based on name, category, price range, and specific shop locations.
- A multi-vendor cart engine that allows customers to add products from different sellers and maintains the cart state even after page refreshes.
- A post-purchase review module allowing customers to provide a single, comprehensive rating and comment for an entire order.
- Administrative tools to manage user accounts and remove products to maintain the security and quality of the platform.

3.Need of the system

- Most local shopkeepers lack the technical expertise or resources to build their own individual websites. This system provides them with a ready-made digital infrastructure to go online instantly.
- Small-scale vendors are losing customers to massive global platforms. DukaanSetu gives them a fighting chance by offering local customers the same convenience of online browsing and home delivery.
- Customers often don't know what products are available in shops just a few blocks away. The system satisfies the need for a "digital window" where users can see local inventory without physically visiting every store.
- Traditional shopkeepers often rely on paper registers to track sales and profits, which is prone to errors. The system automates sales tracking, profit calculation, and order management.
- Today’s shoppers prefer the speed of digital searching and filtering. The system meets this demand by allowing users to compare prices and categories across multiple local shops from a single screen.
- By encouraging people to buy from neighborhood stores rather than distant warehouses, the system helps keep money circulating within the local community, supporting small-scale entrepreneurs.
- There is a need for a platform where a customer can buy groceries from one local shop and electronics from another in a single, unified transaction, which is exactly what this system provides.

4.Module Specification

1. **User Authentication & Identity Module**

- **Registration:** Secure signup for Customers, Shopkeepers, and Admins.
- **Secure Login:** JWT (JSON Web Token) based authentication with encrypted password storage using Bcrypt.js.
- **Role-Based Access Control (RBAC):** Middleware that restricts or grants access to specific routes and UI components based on whether the user is an Admin, Shopkeeper, or Customer

.

2. **Customer Experience Module**
   - **Marketplace Engine:** Advanced search and multi-criteria filtering by category, price, and shop location.
   - **Multi-Vendor Cart:** A persistent shopping cart that aggregates items from various shopkeepers into a single checkout flow.
   - **Order History:** A detailed view for customers to track their previous purchases and current order statuses.
   - **Integrated Review System:** Allows customers to provide comprehensive feedback and ratings for their entire order experience.

3. **Shopkeeper Inventory & Fulfillment Module**
   - **Inventory Manager:** Full CRUD (Create, Read, Update, Delete) for product listings, linked to Admin-defined categories.
   - **Orders Dashboard:** Real-time access to incoming customer orders, including delivery addresses and itemized lists.
   - **Fulfillment Toggle:** Ability to update order status from "Pending" to "Delivered" to track completion.
   - **Earnings Tracker:** Automatic calculation of total profit and sales performance.

4. **Administrative Governance Module**
   - **Category Standardization:** Managing the global list of product categories available to all shopkeepers.
   - **Platform Analytics:** High-level monitoring of total system sales, total orders, and active user counts.
   - **Market Trends:** Tracking "Popular Products" based on purchase frequency.
   - **Moderation Tools:** Ability to remove unauthorized products or users to ensure platform security.

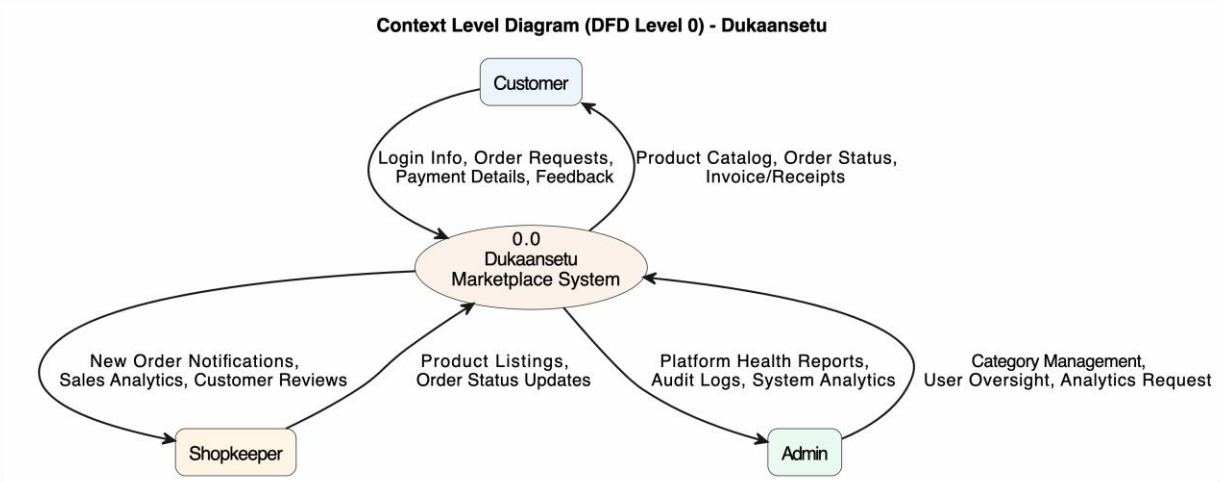
5. **Backend & Data Orchestration Module**
   - **RESTful API Services:** Clean and documented endpoints for frontend-backend communication.

- **Database Management:** Schema definitions for Users, Products, Categories, Orders, and Reviews using MongoDB.
- **Data Population:** Deep-linking logic that connects orders to both products and shopkeepers for accurate reporting.

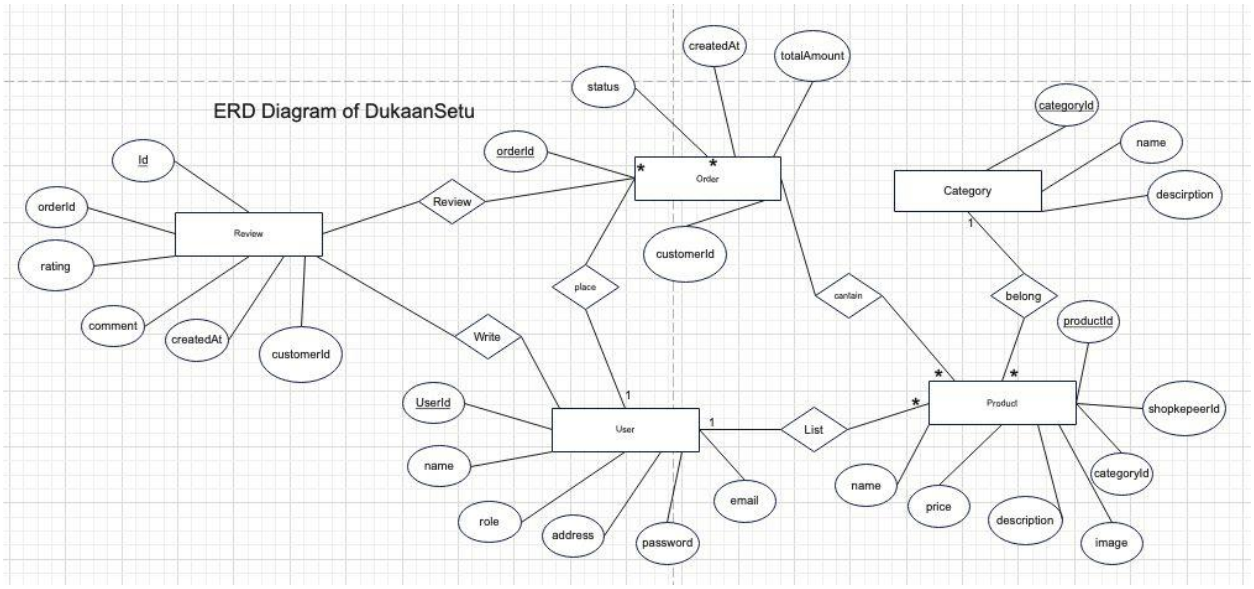
5. **Technology**
   - **Frontend –** Reactjs, Boostrap 5, Css3
   - **Backend -** Nodejs, Bcrypt.js
   - **Database –** MongoDB

CHAPTER 3: ANALYSIS & DESIGN

3.1 Context Level Data Flow Diagram

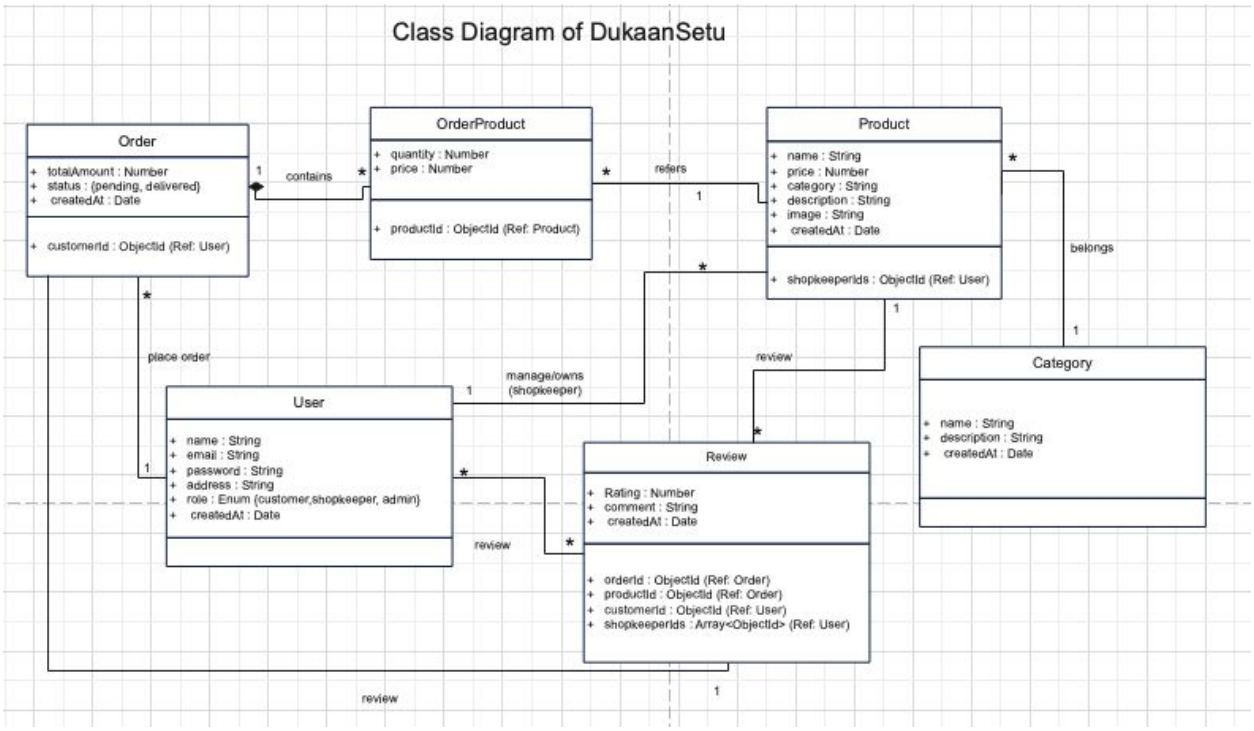


3.2 Entity Relationship Diagram



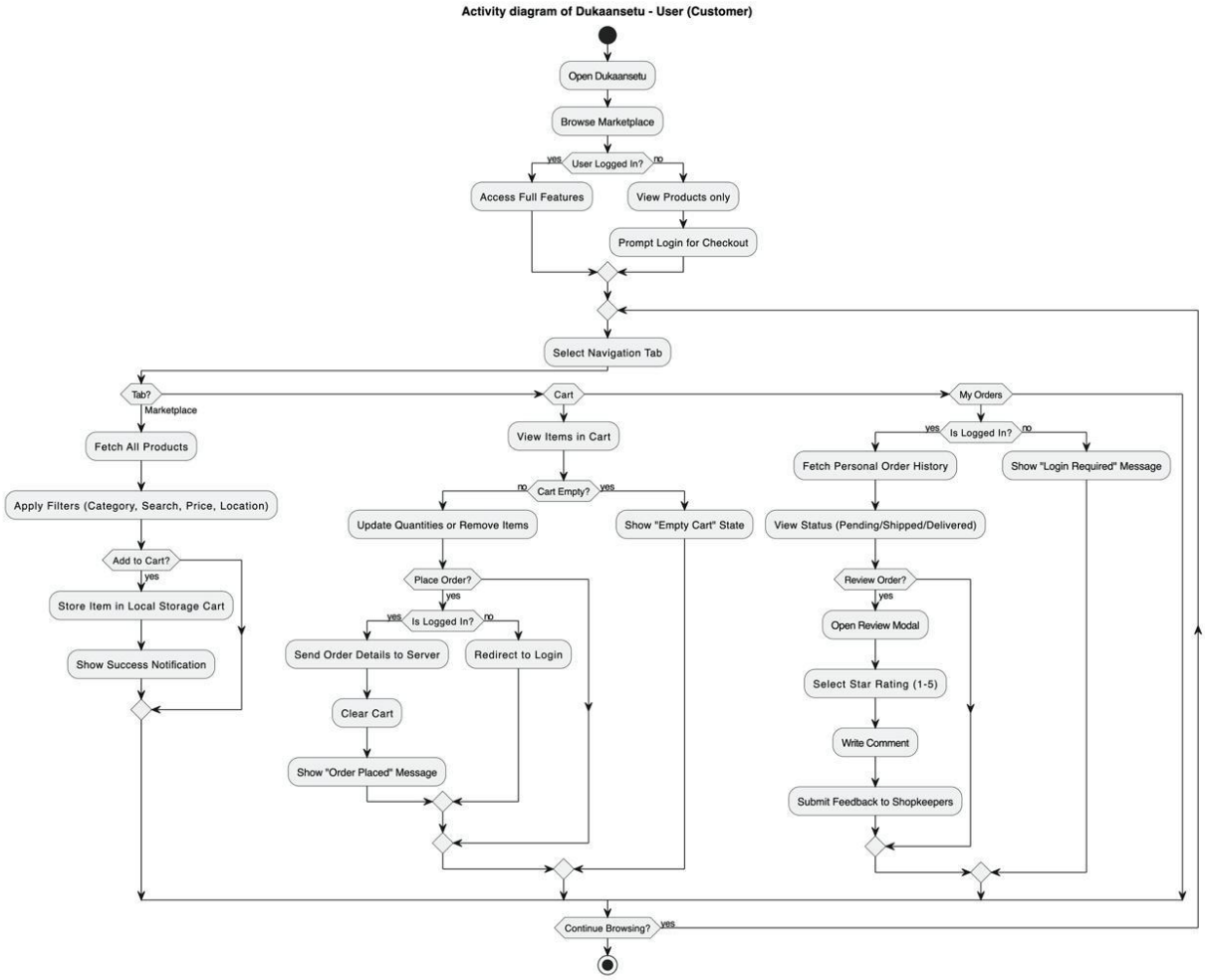
3.3 Use Case Diagram

3.4 Class Diagram

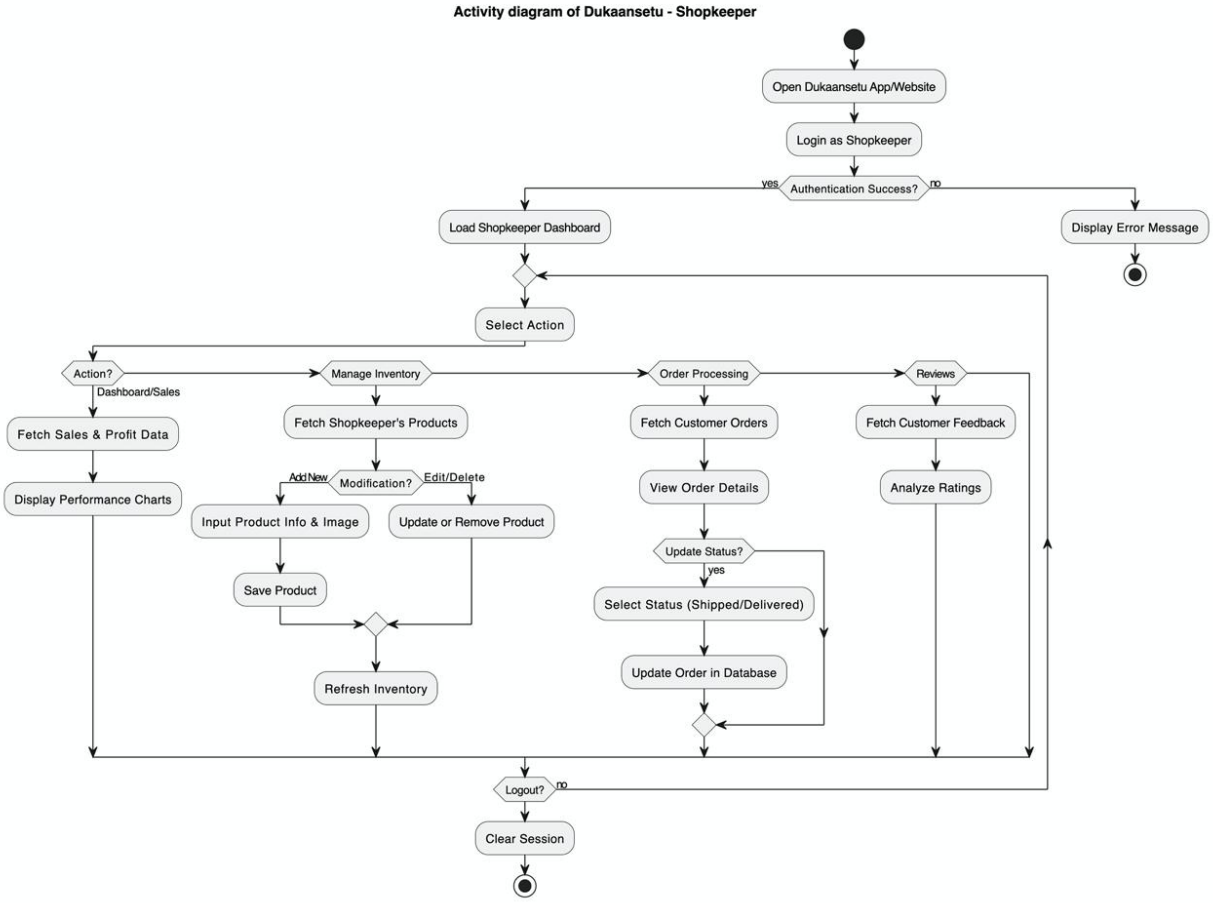


3.5 Activity Diagram

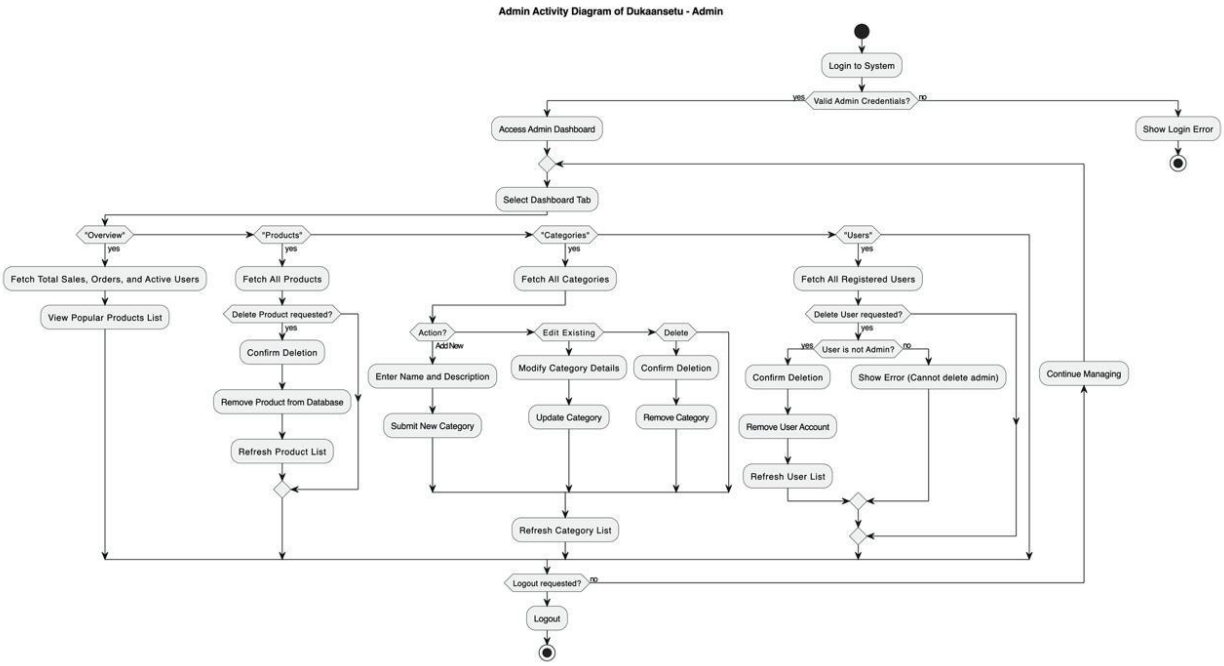
1. User



2. Shopkepper



3. Admin



3.6 Data Dictionary

1. **User Collection**

Field Name	Data Type	Description	Constraints
_id	ObjectId	Unique identifier for each user	Primary Key, Auto-gen
name	String	Full name of the user	Required
email	String	Email address	Unique, Required
password	String	Bcrypt hashed secure password	Required
address	String	delivery address	Required for Orders
role	String	Defines user access	Enum
createdAt	Date	Timestamp	Default : Date.now

2. **Product Collection**

Field Name	Data Type	Description	Constraints
_id	ObjectId	Unique identifier for each user	Primary Key, Auto-gen
name	String	Title of product	Required
Price	String	Cost per unit	Required
category	String	Category name	Defined by admin
description	String	Detail info.	Required

image	String	url of product	Default placeholder
shopkeeperId	ObjcetId	Reference to the user	Foreign key
createdAt	Date	Timestamp	Default : Date.now

3. **Category Collection**

Field Name	Data Type	Description	Constraints
_id	ObjectId	Unique identifier for each user	Primary Key, Auto-gen
name	String	Unique name of category	Required
description	String	Detail info.	Required

4. **Review Collection**

Field Name	Data Type	Description	Constraints
_id	ObjectId	Unique identifier for each user	Primary Key, Auto-gen
OrderId	ObjectId	Specific order reviewed	Foreign key
shopkeeperIds	Array	Ids of all shopkeeper involved order	Foreign key
rating	Number	Numeric score	Min:1 Max:5
customerId	ObjectId	User wrote the review	Foreign key
comment	String	Feedback text	required
createdAt	Date	Timestamp	Default : Date.now

5. **OrderCollection**

Field Name	Data Type	Description	Constraints
_id	ObjectId	Unique identifier for each user	Primary Key, Auto-gen
customerId	ObjectId	Reference of buyer	Foreign key
products	Array	List of items	Non-empty
totalAmount	Number	Total cost	Required
status	String	Current stage	Enum: pending, delivered
createdAt	Date	Timestamp	Default : Date.now

4. **Drawbacks and Limitation**

- The system currently does not facilitate real-time financial transactions, requiring payments to be handled externally or upon delivery.

- Product images are managed through external URLs instead of a direct cloud storage integration (like AWS S3), which may lead to broken links if the source image is removed.
- The application lacks an automated notification engine to alert users of new orders, status changes, or feedback via Email or SMS.
- There is no automated "low-stock" alert system, forcing shopkeepers to manually track their inventory levels and update listings.
- The user interface is currently restricted to the English language, which may limit adoption by local shopkeepers in non-English speaking regions.

5. **Proposed Enhancement / Future Scope**

- Future iterations will include the integration of real-time payment processors such as Razorpay,Stripe, or PayPal to facilitate secure online transactions and automated payouts to shopkeepers.
- Implementing a direct image upload system using services like Cloudinary or AWS S3 will allow shopkeepers to host product images securely on the platform without relying on external URLs.
- To increase accessibility for diverse local markets, the platform will implement localization (i18n) to support multiple regional languages for both the interface and product details.
- Enhancing the Shopkeeper Dashboard with graphical data visualization (charts and graphs) to track monthly sales trends, inventory turnover rates, and low-stock alerts.
- Implementing a real-time chat feature between customers and shopkeepers to allow for pre-purchase inquiries,order customizations, and personalized customer support.

6.**Conclusion**

The development of **DukaanSetu** represents a significant step toward the digital transformation of hyper-local commerce by providing a robust and accessible marketplace for traditional offline shopkeepers. Through the successful implementation of the **MERN stack**, the platform effectively bridges the gap between neighborhood vendors and modern consumers, offering a seamless and secure transaction experience. By introducing specialized,role-based dashboards for Admins, Shopkeepers, and Customers, the system ensures structured platform governance, transparent order fulfillment, and an intuitive product discovery process. Ultimately, the project fulfills its core mission of digital empowerment—allowing local entrepreneurs to digitize their inventory and scale their reach while providing consumers with the convenience of online shopping combined with the trust of their local community stores. As the platform continues to evolve with future enhancements like integrated payments and real-time logistics, it stands poised to revitalize local economies and redefine the neighborhood shopping experience for the digital age.

7.**Bibliography**

- **React.js Documentation:** https://react.dev/

- **Node.js API Reference:** https://nodejs.org/api/
- **Express.js Framework Guide:** https://expressjs.com/
- **MongoDB Manual:** https://www.mongodb.com/docs/manual/
- **Bootstrap Documentation:** https://getbootstrap.com/
- **Axios Library Repository:** https://github.com/axios/axios(For handling asynchronous API requests).